Разные методы генерации из диффузионных моделей

Промежуточный отчёт. Этап 1 (рубежный контроль)

1. Постановка задачи

Исследуется безусловная генерация изображений CIFAR-10 с замороженной DDPM. Меняются численный сэмплер и расположение уровней шума. Цель проекта - найти компромисс между качеством изображений и вычислительной стоимостью при одинаковом бюджете обращений к нейросети.

На первом этапе реализованы DDIM, многошаговый DPM-Solver++ 2M второго порядка и Heun в координатах EDM, выполнены численные проверки, воспроизведён библиотечный baseline и получено предварительное сравнение трёх методов. Дальнейший исследовательский вопрос - снижает ли подбор расписания MSE к длинной генерации на новых шумах и сопровождается ли это улучшением FID.

Ресурсы и ограничения. Основные измерения выполнены на Tesla T4 в Google Colab. Используются float32 и batch size 64. Пик CUDA allocated для baseline составил 0,834 GiB. Квота Colab ограничивает длительные переборы. Локальная генерация поддерживается на M4 Pro через MPS, расчёт FID в этом режиме выполняется на CPU. Время и окружение каждого запуска фиксируются в manifest.

2. Данные и предобученная модель

CIFAR-10 содержит 60 000 цветных изображений 32×32 десяти классов. Официальное разбиение включает train из 50 000 и test из 10 000 изображений. Для FID выбрана реальная выборка train-50k. Генерация безусловная, метки классов в сэмплировании не используются. Источник данных и технический отчёт указаны в [1].

Выбрана google/ddpm-cifar10-32, ревизия `267b167dc01f0e4e61923ea244e8b988f84deb80`. UNet2DModel содержит 35 746 307 параметров и предсказывает epsilon. Модель соответствует заданию и позволяет многократно сравнивать сэмплеры на доступных ресурсах. Используются готовые веса без дообучения. [2]

Объект	Источник и условия
CIFAR-10	Официальная страница требует цитировать Krizhevsky (2009). Отдельная лицензия на этой странице не указана. [1]
Веса DDPM	В текущей карточке модели заявлена Apache License 2.0. [2]
Diffusers 0.35.2	Apache License 2.0. [8]
torch-fidelity 0.3.0	Apache License 2.0. [9]

Код проекта и инструкция воспроизведения доступны в репозитории [10]. Проверенные результаты и планы следующих запусков обозначены отдельно по тексту.

3. Воспроизведение baseline

Точкой отсчёта служит DDIMScheduler из Diffusers 0.35.2 с готовой DDPM. Сгенерированы 5000 изображений из сохранённого CPU-банка шумов. Для изображения i используется seed 50000 + i. Настройки задаются configs/baseline_ddim_100.json и runtime-конфигурацией ноутбука 01.

Условия. DDIM, eta=0, 100 шагов, native leading, clipping=True, float32, batch size 64. Первая точка t=990, последний запрос t=0, затем чистая конечная точка. PyTorch 2.11.0+cu130, CUDA 13.0, Python 3.13.16, Tesla T4. TF32 отключён.

Показатель	Значение	Показатель	Значение
FID-5k / real-50k	15,6571	Изображений	5000
NFE на изображение	100	Batch size	64
Денойзинг, с	1505,17	Секунд на изображение	0,3010
Эквивалентных оценок сети	500 000	Forward для батчей	7900
Пик CUDA allocated	0,834 GiB	Тензоры и PNG	Сохранены

Время представляет сумму измерений денойзинга, включая подготовку сетки и шаги сэмплера. Загрузка модели, прогрев, переносы входа/выхода, запись файлов и FID исключены из этого измерения. Один прогревочный батч выполняется при каждом вызове генерации. Для FID используется torch-fidelity 0.3.0.

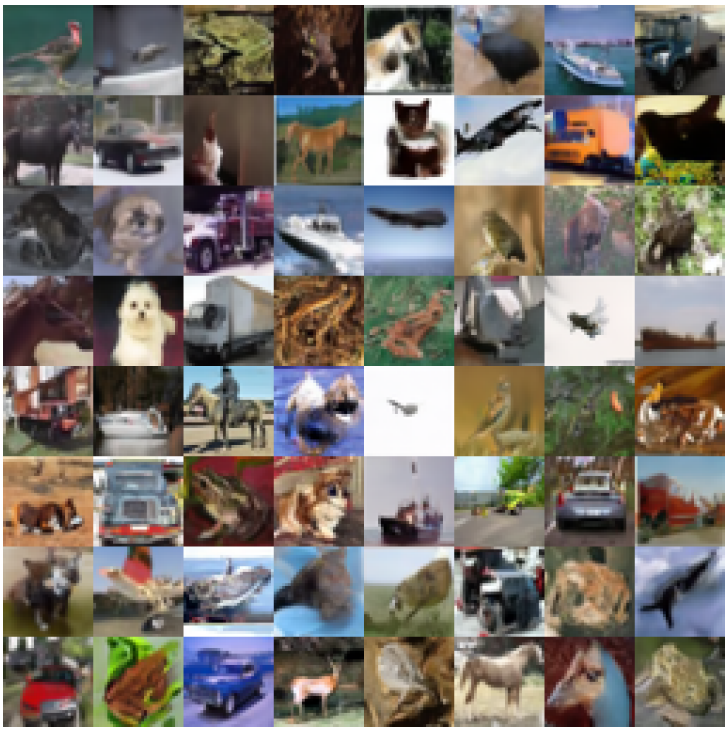


Рисунок 1. Первые 64 изображения библиотечного DDIM-100 из сохранённого preview. Исходный размер каждой картинки 32×32, увеличение методом ближайшего соседа.

Визуальная проверка. В сетке различимы животные, автомобили, самолёты и суда, разнообразны фоны и цвета. На части изображений размыты границы и искажены детали. Такой характер изображений согласуется с низким разрешением CIFAR-10. Численный результат относится к зафиксированному протоколу FID-5k/real-50k.

Полный экспорт содержит шум, сырые тензоры, PNG, конфигурацию, версии, исходники, FID и логи. Проверены контрольные суммы, последовательность 5000 изображений и соответствие PNG тензорам. Baseline завершён и сохранён.

4. Разбор методов и интерфейса модели

Обозначим A_t накопленным произведением коэффициентов $(1 - \beta_u)$ обучающего DDPM-расписания. Коэффициенты a_t и b_t задают масштабы сигнала и шума. [3]

$$x_t = a_t x_0 + b_t \varepsilon,\ a_t = \sqrt{A_t},\ b_t = \sqrt{(1 - A_t)}.$$

Вход выбранной U-Net имеет форму [B, 3, 32, 32] и сопровождается дискретным timestep. Сеть возвращает epsilon. Обозначим D_t оценку чистого изображения. Для разных типов предсказания обвязка имеет следующий вид.

Предсказание	Восстановление чистого изображения и шума
epsilon	$D_t = (x_t - b_t \varepsilon_\theta)/a_t$
x_0	D_t равна выходу сети, $\varepsilon = (x_t - a_t D_t)/b_t$
$v = a_t \varepsilon - b_t x_0$	$D_t = a_t x_t - b_t v_\theta$, $\varepsilon = b_t x_t + a_t v_\theta$

В реализованном checkpoint используется epsilon-параметризация. Преобразования epsilon/x_0/v отдельно проверены на состояниях с известными чистым изображением и шумом. Чистые изображения нормированы в диапазон [-1, 1], зашумлённый вход имеет неограниченный диапазон.

4.1. Детерминированный DDIM

При eta=0 следующий уровень s вычисляется по текущему предсказанию. На один переход требуется один новый forward сети. [4]

$$D_t = (x_t - b_t \varepsilon_\theta)/a_t,\ x_s = a_s D_t + b_s \varepsilon_\theta.$$

При чистом конце a_s=1, b_s=0 результатом является D_t. Собственная реализация шага написана явно. Native-режим согласует clipping с библиотечным DDIM, общие межметодные эксперименты используют clipping=False.

4.2. DPM-Solver++ 2M второго порядка

Многошаговый вариант использует текущую и предыдущую оценки D. Определим $\lambda = \ln(a/b)$, $h = \lambda_s - \lambda_t$, $r = (\lambda_t - \lambda_{prev})/h$. Поправка midpoint и переход имеют вид [5]

$$D_{corr} = D_t + (D_t - D_{prev})/(2r),$$
$$x_s = (b_s/b_t)x_t - a_s(\exp(-h) - 1)D_{corr}.$$

История повышает порядок при одном новом forward на узел. Первый шаг выполняется без истории. На чистом терминальном переходе используется предел первого порядка $x_s = D_t$. Разность exp(-h)-1 вычисляется через expm1.

4.3. Heun в координатах EDM

$$y = x_t/a_t,\ \sigma = b_t/a_t,\ D(y,\sigma) = y - \sigma \varepsilon_\theta,\ dy/d\sigma = (y - D(y,\sigma))/\sigma.$$

Перед вызовом DDPM состояние преобразуется как $x_t = y/\sqrt{(1+\sigma^2)}$. Heun делает пробный Euler-шаг и корректирует его средним двух наклонов. На последнем переходе к σ=0 применяется Euler-предел без второго запроса. Для N ненулевых узлов NFE=2N-1. Stochastic churn отключён. [6]

4.4. Проверка реализации и предварительный эксперимент

Во всех общих сетках используются одинаковые сохранённые z, начало t=999, последний запрос t=0 и чистый конец σ=0. Для Heun начальное состояние $y=\sqrt{(1+\sigma^2)}z$ соответствует тому же VP-состоянию. Библиотечные адаптеры используют эти же соглашения.

В Colab успешно выполнены 35 тестов сохранённой версии пакета и пять численных сверок с Diffusers на двух фиксированных шумах. Сравниваются промежуточные состояния и фактический NFE. Ниже приведены измеренные максимальные ошибки.

Метод / сетка / clipping	NFE	Макс. ошибка шага
DDIM / native / True	20	$2{,}21 \times 10^{-6}$
DPM-Solver++ 2M / karras / False	19	$5{,}72 \times 10^{-6}$
Heun / karras / False	19	0
DPM-Solver++ 2M / log_sigma / True	19	$3{,}77 \times 10^{-6}$
Heun / log_sigma / True	19	0

Дополнительно получены по 5000 изображений трёх собственных методов при 39 NFE, uniform_t, clipping=False, float32, batch size 64 на T4. У DDIM и DPM-Solver++ 39 узлов, у Heun 20 узлов.

Метод	FID ↓	Денойзинг, с	Секунд/изображение
DDIM	20,8112	548,78	0,1098
DPM-Solver++ 2M	18,7129	558,05	0,1116
Heun	27,0767	558,56	0,1117



Рисунок 2. Одинаковые шумы по колонкам, методы по строкам. Номера сверху обозначают индексы изображений. Показаны 8 примеров из визуально просмотренных 64 индексов, равномерно распределённых по набору.

У DPM-Solver++ 2M на ряде примеров отчётливее контуры, например у судна 1349 и автомобиля 4444. У Heun заметнее зернистость на небе 555 и фоне 635. У всех методов встречаются размытые детали и неоднозначные формы. Минимальный FID в этом прогоне получен DPM-Solver++ 2M.

Все 15 000 PNG сопоставлены с тензорами, банки шумов совпали побайтно, проверены SHA256 и итоговые таблицы. Выводы ограничены одним банком и uniform_t. Библиотечный baseline из раздела 3 имеет другой протокол clipping и начала траектории.

5. План экспериментов

Следующие эксперименты запланированы для этапа 2. Сначала сравниваются стандартные расписания, затем реализуется подбор опорных уровней шума.

Ось	Конфигурации и назначение
Сэмплер	DDIM, DPM-Solver++ 2M, Heun
Вычислительный бюджет	19, 39, 79 NFE на изображение
Расписание	uniform_t, log_sigma, karras с rho=7
Качество и точность	FID, визуальная оценка, MSE к длинному эталону
Стоимость	Время денойзинга, NFE, CUDA allocated; отдельно время/NFE поиска
Переносимость	Новые банки шумов и другой сэмплер при том же бюджете

Полная сетка содержит 3×3×3=27 конфигураций. Три uniform_t/39 NFE уже выполнены. Расширение включает ещё шесть конфигураций расписаний при 39 NFE, затем добавляются бюджеты 19 и 79. Для каждого полного FID-прогона используются 5000 изображений. Ресурсный бюджет уточняется по первому батчу.

uniform_t распределяет точки по обучающему timestep, log_sigma по логарифму физического уровня шума, karras по степенному закону. Фактические точки квантуются в обучающую сетку. Контролируются одинаковые края, порядок, уникальность точек и реальный NFE.

5.1. Планируемый подбор опорных уровней

Начальная конфигурация - DPM-Solver++ 2M при 39 NFE с log_sigma. Фиксируются крайние уровни и три внутренние опоры при u=0,25; 0,50; 0,75. Между опорами линейно интерполируется log(sigma), получаются 39 узлов.

Шаг поиска. По очереди каждая внутренняя опора сдвигается на ±δ в log(sigma), начальное δ равно 1/20 диапазона log(sigma). Кандидаты сохраняют порядок опор. После квантования отвергаются повторяющиеся timestep. Выбирается сдвиг с меньшей MSE на 64 calibration-шумах. При равной MSE сохраняется текущая точка.

Остановка и выбор. Рассматриваются максимум 24 различных кандидата, по два направления для трёх опор в четырёх проходах. После прохода без улучшения δ уменьшается вдвое. Три лучших найденных варианта проверяются на 64 validation-шумах. Финальное расписание фиксируется по validation.

Эталон и цель. Детерминированный DDIM-1000 создаётся на тех же шумах. На 16 debug-шумах уже измерена MSE DDIM-500 к DDIM-1000: $5{,}23\times10^{-6}$. Для коротких методов при 39 NFE ошибки к DDIM-1000 составили $2{,}20\times10^{-3}$ (DDIM), $3{,}23\times10^{-4}$ (DPM++ 2M), $9{,}57\times10^{-4}$ (Heun). Это предварительная проверка на малой выборке. Ошибка вычисляется по сырым float32-тензорам. Сохраняются история кандидатов, MSE, время и NFE поиска.

Гипотеза. Увеличение плотности шагов в области средних и низких уровней шума может уменьшить ошибку восстановления структуры и деталей при фиксированном NFE. Положение полезной области и связь с FID будут проверены экспериментально.

Критерий успеха. Уменьшение MSE на независимых шумах при равном NFE, подтверждённое улучшением FID для вывода о качестве изображений. Отдельно оцениваются стоимость поиска и перенос на другой сэмплер. При переносе одни опоры формируют 39 узлов DDIM или 20 узлов Heun для бюджета 39 NFE.

6. Метрики и протокол оценки

FID. Используется torch-fidelity 0.3.0, inception-v3-compat, 2048 признаков. Входные generated - RGB PNG uint8 32×32, resize и нормализация выполняются внутри библиотеки. Сравнение generated-5k с CIFAR-10 train-50k. Сохраняются версия, окружение и хеш точного набора PNG. [7,9]

NFE и время. Счётчик forward проверяет реальный бюджет на изображение. Денойзинг синхронизируется с устройством. Для сравнения времени фиксируются GPU/backend, версии, точность, batch size и прогрев. Поиск расписания измеряется отдельно.

MSE и повторы. Средняя квадратичная ошибка вычисляется по каналам и пикселям каждого float32-изображения, затем усредняется по шумам. Сохраняются индивидуальные ошибки и стандартная ошибка среднего. Выполненные FID-результаты получены по одному прогону на общем банке. Для выбранных конфигураций запланированы два новых банка.

Назначение банка	Размер	Seeds / статус
Debug эталона	16	10000-10015 / выполнено в 04
Calibration поиска	64	20000-20063 / план
Validation выбора	64	30000-30063 / план
Независимая MSE	1024	40000-41023 / план
Основной FID	5000	50000-54999 / выполнено
Подтверждающий FID	2×5000	60000-64999 и 70000-74999 / план

7. План этапа 2 и риски

Последовательность работ - проверка длинного эталона, таблица стандартных сеток, координатный поиск опор, независимые MSE/FID и перенос расписания. Для подтверждения стандартных сеток фиксируются uniform_t-контроли и лучшее расписание каждого метода при 39 NFE, максимум шесть конфигураций.

Ожидаются таблицы FID/NFE/времени, графики компромиссов, история поиска и оценки разброса на новых банках. Основные риски - квота T4, длительный FID на CPU, чувствительность к эталону и переобучение расписания на calibration. Предусмотрены блоки с checkpoint, точные банки шумов, отдельный validation и фиксация конфигураций до независимой проверки. Полный поиск опор ещё предстоит реализовать.

Источники и код

[1] Krizhevsky A. Learning Multiple Layers of Features from Tiny Images. 2009. Официальная страница CIFAR-10/100. cs.toronto.edu/~kriz/cifar.html

[2] Google. ddpm-cifar10-32. Карточка модели и Apache-2.0. huggingface.co/google/ddpm-cifar10-32

[3] Ho J., Jain A., Abbeel P. Denoising Diffusion Probabilistic Models. 2020. arXiv:2006.11239

[4] Song J., Meng C., Ermon S. Denoising Diffusion Implicit Models. 2020. arXiv:2010.02502

[5] Lu C. et al. DPM-Solver++: Fast Solver for Guided Sampling of DPMs. 2022. arXiv:2211.01095

[6] Karras T. et al. Elucidating the Design Space of Diffusion-Based Generative Models. 2022. arXiv:2206.00364

[7] Heusel M. et al. GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium. 2017. arXiv:1706.08500

[8] Hugging Face Diffusers v0.35.2. Код и Apache License 2.0. github.com/huggingface/diffusers

[9] Obukhov A. et al. torch-fidelity v0.3.0. Код и Apache License 2.0. github.com/toshas/torch-fidelity

[10] Код проекта. Ветка codex/baseline, опубликованный коммит 95071f3. github.com/maria-igntv/diffusion-sampling-mipt

Источники и лицензии проверены 05.11.2026. Численные результаты взяты из сохранённых summary.json, fid.json и comparison.json выполненных запусков.